

Fase 3: Transfer

Objetivo

Validar de forma exhaustiva que, vía API, un consumidor puede:

- arrancar una transferencia con un contrato ya acordado;
- consultar su estado y su ciclo de vida;
- recuperar el endpoint y token de acceso mediante EDR;
- y verificar que el recurso puede consumirse realmente desde el data plane.

Alcance

Esta fase **no cubre negociación**. Da por hecho que se llega desde la fase 2 con el `contractId` ya resuelto y con el resto de identificadores necesarios. El propio `TransferRequestDto` documentado exige `connectorId`, `counterPartyAddress`, `contractId`, `assetId`, `protocol`, `transferType` y, opcionalmente, `dataDestination` y `managedResources`.

Idea clave

Aquí el criterio fuerte no es solo “la transferencia se crea”, sino distinguir tres niveles:

- **la transferencia se lanza;**
- **la transferencia progresa / queda registrada;**
- **la transferencia sirve de verdad para acceder al recurso.**

La documentación permite validar esas tres capas porque separa `transferprocesses`, `edrs` y acceso al `publicApiUrl` del dataplane.

Precondiciones

Antes de empezar esta fase, conviene dejar cerradas estas cuatro precondiciones:

1. JWT válido

Las llamadas al Management API y a muchos endpoints de catálogo, negociación y transferencia usan `Authorization: Bearer <JWT>`, obtenido de

Keycloak con `client_id = dataspace-users`.

2. Contrato ya negociado

La lógica funcional general documentada es: catálogo → selección de oferta → negociación/aceptación → acuerdo → transferencia/acceso. Por tanto, transfer no arranca desde cero: necesita ese acuerdo previo.

3. Identificadores ya conocidos

Como mínimo se necesitan: `contractId`, `assetId`, `counterPartyAddress`, `connectorId` y `protocol`. Eso sale directamente del `TransferRequestDto` documentado.

4. Data plane registrado y operativo

El conector debe tener al menos un dataplane registrado, porque su `publicApiUrl` es justo lo que aparece como destino en el EDR. En este despliegue el endpoint `GET /management/v2/dataplanes` **no está expuesto** (devuelve HTTP 404), así que como precheck práctico se sustituye esa comprobación por el smoke test del paso 3.1 (`POST /management/v3/assets/request`). Si el conector está sano y el agreement existe, el EDR del paso 3.4 da por defecto la URL del dataplane usable.

Propuesta cerrada: Fase 3 Transfer por API

Bloque 0. Prechecks técnicos

0.1. Login

Primero se obtiene el JWT desde Keycloak y se verifica que responde una llamada protegida sencilla. La documentación usa ese mismo flujo para Management API y otros endpoints protegidos.

0.2. Confirmar dataplane (indirecto)

En este despliegue `GET /management/v2/dataplanes` devuelve 404, así que la comprobación del dataplane se hace **indirectamente**: si el smoke `POST /management/v3/assets/request` responde JSON y el EDR del paso 3.4 devuelve un `endpoint` válido, el dataplane está operativo. Esa URL del EDR es la que el documento describe como `publicApiUrl` del dataplane.

0.3. Confirmar acuerdo previo

Si en Fase 2 se localiza o produce un agreement, conviene dejar anotado el `contractId` exacto con el que se disparará la transferencia. La Count API también contempla `type=contractAgreement`, así que puede usarse como control auxiliar antes de empezar.

Bloque 1. Arranque de la transferencia

Este es el corazón del flujo. El conector documenta dos endpoints v3 para iniciar transferencia: `POST /v3/inesdatatransferprocesses` como alias específico y `POST /v3/transferprocesses` como estándar EDC. En la colección de entorno local se usa `POST /management/v3/transferprocesses`, que devuelve un `IdResponse` con el `transferId`.

Endpoint principal

`POST /management/v3/transferprocesses`

Payload base a validar

Debería incluir, como mínimo:

- `connectorId`
- `counterPartyAddress`
- `contractId`
- `assetId`
- `protocol`
- `transferType`

y opcionalmente:

- `dataDestination`
- `managedResources`

Qué validar aquí

Se recomienda probar al menos estos casos:

Caso feliz mínimo

Lanzar una transferencia con el payload mínimo correcto y comprobar que se recibe `transferId`.

Caso con destino explícito

Si el asset o escenario lo requiere, probar con `dataDestination` para ver si el proceso cambia de comportamiento o validación. La documentación lo contempla como campo opcional.

Caso con `managedResources`

Probar si el conector acepta ese parámetro y qué efecto tiene. La documentación lo lista en el DTO, así que merece una prueba específica.

Qué debe salir de este bloque

No basta con un `200`. Deben guardarse:

- `transferId`
 - payload exacto usado
 - `contractId`
 - `assetId`
 - provider / `counterPartyAddress`
 - timestamp de inicio
-

Bloque 2. Seguimiento del proceso

Una vez creado el proceso, el core expone endpoints para listar transferencias, obtener detalle, consultar estado y controlar el ciclo de vida. Eso ya proporciona una validación bastante más rica que la interfaz, que según las notas disponibles solo mostraba tablas resumen en contracts y transfer history.

Endpoints clave

- `POST /management/v3/transferprocesses/request`
- `GET /management/v3/transferprocesses/{id}`
- `GET /management/v3/transferprocesses/{id}/state`

Qué validar aquí

1. Recuperación individual

Nada más crear la transferencia, hacer `GET /management/v3/transferprocesses/{transferId}` y comprobar que el proceso existe de

verdad.

2. Estado

Usar `GET /management/v3/transferprocesses/{transferId}/state` para medir si hay evolución temporal del proceso y no solo alta inicial.

3. Búsqueda / listado

Con `POST /management/v3/transferprocesses/request` validar que la transferencia aparece también en búsquedas o listados, no solo por id.

4. Conteo

Usar `POST /management/pagination/count?type=transferProcess` antes y después para comprobar que el alta del proceso impacta realmente en el estado global del sistema.

Qué se busca confirmar

Aquí conviene separar tres posibles situaciones:

- el `POST` devuelve `transferId`, pero luego el proceso no existe;
- el proceso existe, pero no progresa;
- el proceso progresa, pero luego no produce EDR ni acceso.

La API documentada permite distinguir esas tres cosas bastante bien.

Bloque 3. Ciclo de vida y control

El conector documenta operaciones de control sobre el transfer process: `suspend`, `resume`, `deprovision` y `terminate`. Esto permite ir más allá del happy path y validar robustez operativa real.

Endpoints de control

- `POST /management/v3/transferprocesses/{id}/suspend`
- `POST /management/v3/transferprocesses/{id}/resume`
- `POST /management/v3/transferprocesses/{id}/deprovision`
- `POST /management/v3/transferprocesses/{id}/terminate`

Qué validar

Suspend / resume

Confirmar si el proceso acepta suspensión y reanudación, y si el estado reflejado después cambia de forma coherente.

Terminate

La colección local documenta explícitamente `terminate`, así que se recomienda usarlo como prueba obligatoria para cerrar un proceso y ver si desaparece, cambia de estado o queda trazabilidad residual.

Deprovision

Aquí merece la pena comprobar si está disponible en el despliegue y qué hace realmente sobre un proceso ya materializado. El endpoint está documentado, pero su semántica exacta conviene confirmarla empíricamente.

Casos negativos útiles

- suspender un proceso inexistente;
- reanudar uno no suspendido;
- terminar dos veces el mismo;
- operar sobre un `transferId` mal formado.

La documentación no enumera todos los códigos de error esperables, así que esta parte sirve justo para mapear el comportamiento real del conector.

Bloque 4. EDR y acceso real al recurso

Este es el bloque decisivo. Para consumir datos tras una transferencia, el consumidor pide al conector la dirección del endpoint de lectura y el token mediante:

```
GET /management/v3/edrs/{transferId}/dataaddress
```

(En la colección Postman antigua el path es `v1`, pero en este despliegue solo responde `v3`; `v1` y `v2` devuelven HTTP 404.)

La respuesta devuelve `authorization` y la URL destino, que corresponde al `publicApiUrl` del dataplane. Después, el consumo real se hace con un `GET` al endpoint público del data plane usando el token en `Authorization: Bearer <token>` (en este despliegue el campo `authorization` viene como JWT crudo, por lo que es necesario prefijar `Bearer` manualmente).

Endpoints clave

- `GET /management/v3/edrs/{transferId}/dataaddress` (en este despliegue; `v1` y `v2` devuelven 404)
- `POST /management/v3/edrs/request` con `QuerySpec` para listar EDRs disponibles
- `GET <publicApiUrl>` con `Authorization: Bearer <authorizationToken>` (añadir el prefijo `Bearer` manualmente, ver paso 3.5)

Qué validar aquí

1. Disponibilidad del EDR

Comprobar si, tras lanzar la transferencia, el `transferId` ya tiene EDR resoluble o si hace falta esperar a cierto estado. La documentación no fija el momento exacto, así que esto es una prueba muy valiosa.

2. Coherencia del destino

Verificar si la URL devuelta por EDR coincide con el `publicApiUrl` del dataplane registrado. Esto es una comprobación directa de consistencia interna del despliegue.

3. Consumo real

Usar el token del EDR para hacer una petición al endpoint público del data plane y comprobar si el recurso responde realmente. Ese es el criterio más fuerte de éxito de toda la fase.

Qué se busca distinguir

Aquí pueden encontrarse tres escenarios:

- **transferencia creada pero sin EDR;**
- **EDR devuelto pero acceso fallido;**
- **EDR devuelto y acceso correcto.**

Esa clasificación permite separar muy bien si el fallo está en el proceso de transferencia, en el dataplane o en la exposición final del recurso.

Bloque 5. Robustez y casos límite

Para una validación completa, no conviene quedarse solo en el camino feliz.

Pruebas recomendadas

Duplicados

Lanzar dos veces transferencia sobre el mismo `contractId` y `assetId` para ver si el conector lo permite, lo bloquea o genera procesos independientes.

Parámetros mal formados

Probar `assetId`, `contractId` o `counterPartyAddress` erróneos para ver si el conector valida pronto o falla más tarde. El DTO documenta esos campos como nucleares, así que merecen negativas específicas.

Sin dataplane usable

Si es posible aislar un entorno donde el dataplane esté mal registrado, esta es una prueba muy buena para ver si el proceso se crea igual pero no produce EDR utilizable. Esto está muy alineado con la relación documentada entre dataplane y `publicApiUrl`.

Varios espacios / varios conectores

Repetir exactamente la misma batería en más de un espacio o más de un conector para comprobar si el comportamiento depende del despliegue concreto. El documento insiste en que los conectores son la unidad funcional real del sistema.

Qué sería “probar absolutamente todo” en Transfer por API

Se pueden usar estas siete preguntas de control:

1. Arrancar

¿acepta el `POST /transferprocesses` y devuelve `transferId`?

2. Persistir

¿se puede recuperar el proceso por id y por query?

3. Evolucionar

¿el estado cambia de forma coherente?

4. Contar

¿el contador `type=transferProcess` refleja el alta?

5. Controlar

¿funcionan `suspend`, `resume`, `deprovision` y `terminate`?

6. Resolver EDR

¿el conector devuelve URL y token de acceso?

7. Consumir

¿ese token permite acceder de verdad al recurso publicado?

Orden recomendado de ejecución

Se recomienda este orden:

Ronda 1. Smoke test

- login;
- comprobar dataplane;
- lanzar una transferencia mínima;
- recuperar `transferId`;
- consultar su estado.

Ronda 2. Happy path completo

- `POST /management/v3/transferprocesses` ;
- `GET /transferprocesses/{id}` ;
- `GET /transferprocesses/{id}/state` ;
- `GET /management/v3/edrs/{transferId}/dataaddress` (no `v1`);
- acceso al `publicApiUrl` con `Authorization: Bearer <EDR_TOKEN>` .

Ronda 3. Robustez

- duplicados;
- ids incorrectos;
- terminate;
- suspend/resume;
- varios conectores / varios espacios.

Ronda 4. Auditoría mínima

- count antes y después;
- guardar payload, `transferId`, estado final y resultado del EDR;

- clasificar cada caso como: creado / progresado / accesible.
-

Registro de incidencias recomendado

Para transfer, se recomienda esta ficha:

Módulo

TransferProcess / EDR / DataPlane / Public API

Caso de prueba

nombre corto

Precondición

`contractId`, `assetId`, provider y dataplane disponibles

Endpoint

ruta exacta

Payload

resumen del `TransferRequestDto`

Esperado

qué debía ocurrir

Observado

qué ocurrió

Estado del proceso

respuesta de `/transferprocesses/{id}` o `/state`

EDR

si existe o no, y qué devuelve

Acceso real

si el recurso responde o no

Clasificación

bug / integración / validación insuficiente / configuración

Severidad

bloqueante / alta / media / baja

IDs clave

`contractId` , `assetId` , `transferId`

Evidencia

respuesta HTTP, log o captura

Recomendación práctica

Se recomienda empezar con **un único contrato muy controlado** procedente de la Fase 2. Primero, el camino mínimo:

contractId correcto → **transferprocess** → **state** → **EDR**.

Y después el camino fuerte:

EDR → **publicApiUrl** → **acceso real al recurso**. Eso es lo que realmente cierra la validación de transfer.

Puede prepararse una **checklist operativa completa de las 3 fases por API** en formato paso a paso para Postman o curl.

Comandos del taller

Esta sección contiene los `curl` exactos para arrancar la transferencia, seguir su estado, resolver el EDR y acceder de verdad al recurso. Se asume que la Fase 2: Consumer ha terminado y que `AGREEMENT_ID` , `ASSET_ID` y el resto de variables están cargadas en la terminal.

- `dataspace`
- `suffix`
- `provider`
- `consumer`
- `phase1`
- `phase2`
- `data env`

3.1 Smoke test del management (provider)

Antes de lanzar la transferencia conviene confirmar que el Management API del provider responde y que el `PROVIDER_JWT` sigue válido. Como en este despliegue

`GET /management/v2/dataplanes` devuelve HTTP 404 (el endpoint no está expuesto), el smoke se hace contra `assets/request`, que sí responde con JSON.

```
curl -s -X POST "${PROVIDER_BASE}/management/v3/assets/request" \
  -H "Authorization: Bearer ${PROVIDER_JWT}" \
  -H "Content-Type: application/json" \
  -d '{"@context":{"@vocab":"https://w3id.org/edc/v0.0.1/ns/"},"offset":0,"limit":1,"filterExpression":[]}' \
  | jq .
```



AVISO: Renueva el JWT al empezar Fase 3

Si se viene de otra terminal o ha pasado un rato desde Fase 2, regenerar `PROVIDER_JWT` y `CONSUMER_JWT` antes del smoke (paso descrito en el bloque "Smoke test inicial" de la [Taller APIs - Plan de ejecución](#)). Si el `curl` devuelve HTML en vez de JSON, el JWT ha caducado.



IMPORTANTE: EDR = Endpoint Data Reference

Es el "ticket de acceso temporal" al dato real que se obtiene en el paso 3.4 tras arrancar la transferencia.

Incluye:

- **endpoint:** URL del data plane a la que llamar para leer el recurso.
- **authorization:** credencial temporal (en este despliegue, JWT raw, ver paso 3.5).
- a veces metadatos extra: expiración, `authType`, etc.

3.2 Lanzar la transferencia (HttpData-PULL)

La variante más sencilla con assets HTTP es `HttpData-PULL`: el consumer recibe un EDR con la URL del data plane y un token, y luego hace la lectura directamente. Para escenarios S3 (push) habría que cambiar `transferType` a `AmazonS3-PUSH` o usar el alias `POST /management/v3/inesdatatransferprocesses` con `dataDestination`.

```

TRANSFER_RESPONSE=$(curl -s -X POST "${CONSUMER_BASE}/management/v3/transferprocesses" \
  -H "Authorization: Bearer ${CONSUMER_JWT}" \
  -H "Content-Type: application/json" \
  -d "$(cat <<EOF
{
  "@context": { "@vocab": "https://w3id.org/edc/v0.0.1/ns/"
},
  "@type": "TransferRequest",
  "assetId": "${ASSET_ID}",
  "contractId": "${AGREEMENT_ID}",
  "counterPartyAddress": "${PROVIDER_PROTOCOL}",
  "protocol": "dataspace-protocol-http",
  "transferType": "HttpData-PULL"
}
EOF
)")

echo "${TRANSFER_RESPONSE}" | jq .

export TRANSFER_ID=$(echo "${TRANSFER_RESPONSE}" | jq -r 'if type=="object" then .["@id"] else empty end')
echo "TRANSFER_ID=${TRANSFER_ID}"

```

El `jq` final usa `if type=="object" then .["@id"] else empty end` para no romper si el conector responde con un array de errores en vez de un `IdResponse`.



ERROR: mandatory value '...contractId' is missing or it is blank

Si al lanzar el `TransferRequest` el conector responde:

```
[{"message": "mandatory value 'https://w3id.org/edc/v0.0.1/ns/contractId' is missing or it is blank", "type": "ValidationFailure", "path": "https://w3id.org/edc/v0.0.1/ns/contractId", "invalidValue": null}]
```

La causa prácticamente siempre es que `AGREEMENT_ID` (o `ASSET_ID`) **están vacíos en la terminal**. Pasa cuando se abre una shell nueva para Fase 3 sin reexportar los IDs de Fase 2.

SOLUCIÓN: copiar los valores del resumen del paso 2.7 y reexportarlos. Por ejemplo:

```
export AGREEMENT_ID="<uuid del paso 2.5>"
export ASSET_ID="asset-${SUFFIX}"
export PROVIDER_PROTOCOL="http://${PROVIDER}:19194/protocol"
echo "AGREEMENT_ID=${AGREEMENT_ID}"
echo "ASSET_ID=${ASSET_ID}"
```

Después de eso, el bloque del 3.2 ya toma los valores correctos y la respuesta es `IdResponse` con `@id` válido.

3.3 Polling del estado del transfer process

La transferencia es asíncrona. Se itera hasta llegar a `STARTED` o `COMPLETED`. Si llega a `TERMINATED`, debe revisarse el log del provider.

```
for i in $(seq 1 20); do
  TP_RESPONSE=$(curl -s "${CONSUMER_BASE}/management/v3/transferprocesses/${TRANSFER_ID}" \
    -H "Authorization: Bearer ${CONSUMER_JWT}")
  TP_STATE=$(echo "${TP_RESPONSE}" | jq -r '.state // .edc:state')
  echo "[${i}] transfer state=${TP_STATE}"
  if [ "${TP_STATE}" = "STARTED" ] || [ "${TP_STATE}" = "CO
```

```

MPLETED" ]; then break; fi
    if [ "${TP_STATE}" = "TERMINATED" ]; then echo "Transfer
TERMINATED"; break; fi
    sleep 3
done

curl -s "${CONSUMER_BASE}/management/v3/transferprocesses/
${TRANSFER_ID}/state" \
    -H "Authorization: Bearer ${CONSUMER_JWT}" | jq .

```

3.4 Resolver EDR (endpoint público + token de acceso)

Una vez la transferencia está en **STARTED**, el conector consumer expone un Endpoint Data Reference con la URL pública del data plane y el token de autorización. **El endpoint correcto en este despliegue es v3**, no **v1** como aparece en la colección Postman antigua.

```

EDR_RESPONSE=$(curl -s "${CONSUMER_BASE}/management/v3/edr
s/${TRANSFER_ID}/dataaddress" \
    -H "Authorization: Bearer ${CONSUMER_JWT}")

echo "${EDR_RESPONSE}" | jq .

export EDR_URL=$(echo "${EDR_RESPONSE}" | jq -r '.endpoin
t')
export EDR_TOKEN=$(echo "${EDR_RESPONSE}" | jq -r '.authori
zation')

echo "EDR_URL=${EDR_URL}"
echo "EDR_TOKEN=${EDR_TOKEN:0:30}..."

```



AVISO: ¿HTTP 404 al resolver EDR?

La colección Postman de `inesdata-local-env` documenta el path `GET /management/v1/edrs/{transferId}/dataaddress`. En este despliegue `v1` y `v2` devuelven **404** y solo responde `v3`.

Diagnóstico rápido si aparece HTML con `Error 404 Not Found`:

```
for v in v1 v2 v3; do
  echo "=== ${v} ==="
  curl -s -w "\nHTTP %{http_code}\n" \
    "${CONSUMER_BASE}/management/${v}/edrs/${TRANSFER_ID}/dataaddress" \
    -H "Authorization: Bearer ${CONSUMER_JWT}" | tail -3
done
```

Como alternativa de listado de todos los EDRs disponibles para este consumer:

```
curl -s -X POST "${CONSUMER_BASE}/management/v3/edrs/request" \
  -H "Authorization: Bearer ${CONSUMER_JWT}" \
  -H "Content-Type: application/json" \
  -d '{"@context":{"@vocab":"https://w3id.org/edc/v0.0.1/ns/"},"filterExpression":[]}' \
  | jq '.[] | {transferProcessId, agreementId, assetId, providerId}'
```

3.5 Acceso real al recurso

La prueba definitiva: se hace `GET` al `EDR_URL` con el token y debería recibirse el JSON original de `jsonplaceholder.typicode.com/todos`. Si esto responde, el flujo end-to-end está cerrado.

En este despliegue el campo `authorization` del EDR viene como **JWT raw** (sin prefijo `Bearer`), así que la cabecera se construye con `Authorization: Bearer ${EDR_TOKEN}`.


```
curl -s "${EDR_URL}" \
-H "Authorization: ${EDR_TOKEN}" | jq .
```



ERROR: `{"errors":["Failed to decode token"]}`

Si el dataplane responde ese JSON, el problema está en el header

`Authorization :`

- Si `EDR_TOKEN` empieza por `Bearer` , no debe volver a añadirse el prefijo (`-H "Authorization: ${EDR_TOKEN}"`).
- Si `EDR_TOKEN` viene como JWT crudo (caso de este despliegue), debe añadirse el prefijo manualmente (`-H "Authorization: Bearer ${EDR_TOKEN}"`).

Para comprobarlo rápido:

```
echo "${EDR_RESPONSE}" | jq -r '.authorization' | head -c 10
```

Si imprime `eyJ...` → JWT raw, prefijar con `Bearer` .

Si imprime `Bearer eyJ` → ya viene prefijado, se pasa tal cual.

3.6 Robustez: terminate y duplicado

Dos pruebas rápidas para mapear el comportamiento real del conector frente a operaciones de control y duplicidad. Primero se termina el proceso activo:

```
curl -s -X POST "${CONSUMER_BASE}/management/v3/transferprocesses/${TRANSFER_ID}/terminate" \
-H "Authorization: Bearer ${CONSUMER_JWT}" \
-H "Content-Type: application/json" \
-d '{"@context":{"@vocab":"https://w3id.org/edc/v0.0.1/ns/"},"reason":"taller-cleanup"}' \
-w "\nHTTP %{http_code}\n"
```



IMPORTANTE: Estado `DEPROVISIONED` en `Transfer History` no es un fallo

Tras un `terminate`, el ciclo de vida normal en EDC es `STARTED -> TERMINATED -> DEPROVISIONING -> DEPROVISIONED`. La UI de `Transfer History` mostrará la fila del transfer terminado en estado `DEPROVISIONED` y, si justo después se lanza otra transferencia con el mismo `contractId`, aparecerán **dos filas** para el mismo `Contract ID`:

- una en `STARTED` (la nueva, activa)
- otra en `DEPROVISIONED` (la anterior, ya cerrada)

Es comportamiento esperado: el proceso terminado libera recursos pero queda en historial para auditoría. Para la demo en directo, terminar **una sola vez** y dejar la nueva transferencia en `STARTED` deja en pantalla los dos estados de forma didáctica.

Y después se lanza otra transferencia exactamente igual con el mismo `contractId` y `assetId` para ver si el conector la admite, la encadena o la rechaza:

```
curl -s -X POST "${CONSUMER_BASE}/management/v3/transferprocesses" \
  -H "Authorization: Bearer ${CONSUMER_JWT}" \
  -H "Content-Type: application/json" \
  -d "$(cat <<EOF
{
  "@context": { "@vocab": "https://w3id.org/edc/v0.0.1/ns/"
},
  "@type": "TransferRequest",
  "assetId": "${ASSET_ID}",
  "contractId": "${AGREEMENT_ID}",
  "counterPartyAddress": "${PROVIDER_PROTOCOL}",
  "protocol": "dataspace-protocol-http",
  "transferType": "HttpData-PULL"
}
EOF
)" | jq .
```

3.7 Auditoría final: contadores antes y después

Misma fotografía que en la Fase 1, ahora desde el provider, para validar que el flujo ha dejado huella en `transferProcess` y `contractAgreement`.

```
for type in transferProcess contractAgreement; do
    count=$(curl -s -X POST "${PROVIDER_BASE}/management/pagination/count?type=${type}" \
        -H "Authorization: Bearer ${PROVIDER_JWT}" \
        -H "Content-Type: application/json" \
        -d '{"@context":{"@vocab":"https://w3id.org/edc/v0.0.1/ns/"},"filterExpression":[]}')
    echo "provider ${type}: ${count}"
done

for type in transferProcess contractAgreement; do
    count=$(curl -s -X POST "${CONSUMER_BASE}/management/pagination/count?type=${type}" \
        -H "Authorization: Bearer ${CONSUMER_JWT}" \
        -H "Content-Type: application/json" \
        -d '{"@context":{"@vocab":"https://w3id.org/edc/v0.0.1/ns/"},"filterExpression":[]}')
    echo "consumer ${type}: ${count}"
done
```

3.8 Resumen final del taller

Cómoda referencia con todos los IDs generados a lo largo de las tres fases. Útil tanto para auditoría como para limpieza posterior con la sección **Limpieza opcional al cierre** de la Taller APIs - Plan de ejecución.

```
cat <<EOF

===== TALLER COMPLETO =====
SUFFIX:                ${SUFFIX}
ASSET_ID:              ${ASSET_ID}
ACCESS_POLICY_ID:      ${ACCESS_POLICY_ID}
CONTRACT_POLICY_ID:    ${CONTRACT_POLICY_ID}
CD_ID:                 ${CD_ID}
NEG_ID:                ${NEG_ID}
```

```
AGREEMENT_ID:      ${AGREEMENT_ID}
TRANSFER_ID:       ${TRANSFER_ID}
EDR_URL:           ${EDR_URL}
=====

EOF
```



AVISO: Como se cambia de terminal durante el taller, algunas variables pueden no estar exportadas. En concreto:

```
ACCESS_POLICY_ID:
CONTRACT_POLICY_ID:
CD_ID:
NEG_ID:
```

```
{
  printf 'export SUFFIX=%q\n' "$SUFFIX"
  printf 'export ASSET_ID=%q\n' "$ASSET_ID"
  printf 'export ACCESS_POLICY_ID=%q\n' "$ACCESS_POLICY_ID"
  printf 'export CONTRACT_POLICY_ID=%q\n' "$CONTRACT_POLICY_ID"
  printf 'export CD_ID=%q\n' "$CD_ID"
  printf 'export NEG_ID=%q\n' "$NEG_ID"
  printf 'export AGREEMENT_ID=%q\n' "$AGREEMENT_ID"
  printf 'export TRANSFER_ID=%q\n' "$TRANSFER_ID"
  printf 'export EDR_URL=%q\n' "$EDR_URL"
} > "/Users/jpardo/Desktop/Proyectos/INESdata/data_spaces/demos/API/phase3_env.sh"
```